GENOME-X: Synthetic Genome Editing and Verification Engine

Verilog Quest 2026 | Round 1: Module Design and Verification

Part 1: Mission Brief, DNA Representation and Naming Convention

1.1 Mission Brief

Year: 2089.

A critical system responsible for processing synthetic genetic sequences has suffered a failure. Its sequence-search, editing, and verification subsystems must be rebuilt before the system can be brought back online.

Your mission is to design **GENOME-X**, a digital hardware system that searches a synthetic DNA sequence, identifies a target region, performs a controlled edit, and independently verifies the result.

The system must be designed using synthesizable Verilog, tested through simulation.

1.2 DNA Representation

Each DNA base is represented using two bits.

Base	Encoding
A	`00`
C	`01`
G	`10`

T 11

The system uses the following sequence sizes:

Parameter	Size
Genome	32 bases, represented by 64 bits
Guide sequence	8 bases, represented by 16 bits
Replacement sequence	4 bases, represented by 8 bits
Mismatch threshold	Configurable value from 0 to 2

The genome is packed such that base 0 occupies bits `[63:62]`, base 1 occupies `[61:60]`, and so on.

1.3 Module Naming Convention

Each team must include a unique group identifier in the name of every module it implements. This is mandatory to distinguish modules belonging to different teams during evaluation and integration.

Use the following format:

`<group_id>_<module_name>`

For example, if your group identifier is `T07`, your module names must be:

- `T07_guide_matcher`
- `T07_genome_window_generator`
- `T07_target_locator`
- `T07_genome_editor`

- `T07_edit_verifier`

Replace `T07` with your assigned group identifier. Use the same identifier consistently across all five modules.

Naming requirements:

- Use a short, unique group identifier assigned or approved by the organizers.
- Use lowercase letters, numbers, and underscores only.
- Preserve the required module-name suffix exactly.
- Ensure that all module declarations, instantiations, and testbenches use the same names.
- Do not modify the specified input/output names or widths.

Part 2: Module Specifications I, Guide Matcher and Genome Window Generator

2.1 Module 1: Guide Matcher

Required module name: `<group_id>_guide_matcher`

Objective: Determine whether an 8-base candidate window matches a given guide sequence.

Signal	Direction	Width	Description
`guide`	Input	16 bits	8-base guide sequence
`window`	Input	16 bits	Candidate 8-base window
`threshold`	Input	2 bits	Maximum permitted mismatches, 0–2
`mismatch_count`	Output	4 bits	Number of differing bases

`match`	Output	1 bit	Asserted when mismatch count is within the threshold

Functional requirements:

- Compare corresponding DNA bases, not individual bits.
- Count the number of positions where the bases differ.
- Assert `match` when `mismatch_count` is less than or equal to `threshold`.
- Handle threshold values of 0, 1, and 2 correctly.

2.2 Module 2: Genome Window Generator

Required module name: `<group_id>_genome_window_generator`

Objective: Generate overlapping 8-base windows from consecutive input fragments.

Each incoming fragment contains 8 DNA bases, represented by 16 bits.

Signal	Direction	Width	Description
`clk`	Input	1 bit	System clock
`rst`	Input	1 bit	Synchronous active-high reset
`fragment`	Input	16 bits	Incoming 8-base fragment
`fragment_valid`	Input	1 bit	Indicates a valid incoming fragment
`window`	Output	16 bits	Generated 8-base window

`window_valid`	Output	1 bit	Indicates a valid output window

Functional requirements:

- Accept fragments in sequence.
- Generate consecutive, overlapping 8-base windows across fragment boundaries.
- Preserve the order of bases.
- Avoid losing or duplicating windows.
- Define and implement when the first output window becomes valid.

The behavior and timing of the window generator must be documented clearly in your design explanation.

Part 3: Module Specifications II, Target Locator and Genome Editor

3.1 Module 3: Target Locator

Required module name: `<group_id>_target_locator`

Objective: Search a 32-base genome and identify the first eligible target matching the guide sequence.

Signal	Direction	Width	Description
`clk`	Input	1 bit	System clock
`rst`	Input	1 bit	Synchronous active-high reset
`start`	Input	1 bit	Starts a search

`genome`	Input	64 bits	Original genome
`guide`	Input	16 bits	Guide sequence
`threshold`	Input	2 bits	Maximum permitted mismatches
`target_found`	Output	1 bit	Indicates that an eligible target was found
`target_position`	Output	5 bits	Starting base index of the selected target
`mismatch_count`	Output	4 bits	Mismatch count at the selected target
`done`	Output	1 bit	Indicates that the search has completed

Functional requirements:

- Search the genome from base position 0 through base position 24.
- Examine all valid 8-base windows, including overlapping windows.
- Use the configured mismatch threshold.
- Report the leftmost eligible target, its starting position, and its mismatch count.
- Correctly indicate when no eligible target exists.
- Assert `done` when the search has finished.
- Use the functionality of both the Guide Matcher and Genome Window Generator. Document the interface and method used to connect these functions within the search design.

3.2 Module 4: Genome Editor

Required module name: `<group_id>_genome_editor`

Objective: Perform a controlled substitution of four consecutive bases at a specified position.

Signal	Direction	Width	Description
`clk`	Input	1 bit	System clock
`rst`	Input	1 bit	Synchronous active-high reset
`edit_request`	Input	1 bit	Requests an edit
`genome_in`	Input	64 bits	Original genome
`edit_position`	Input	5 bits	Starting base index of the edit
`replacement`	Input	8 bits	Four-base replacement sequence
`genome_out`	Output	64 bits	Resulting genome
`edit_success`	Output	1 bit	Indicates a successful edit
`done`	Output	1 bit	Indicates request completion

Functional requirements:

- Replace exactly four consecutive bases beginning at `edit_position`.
- Accept only starting positions from 0 through 28, inclusive.

- Preserve all bases outside the selected four-base region.
- Reject invalid edit positions.
- Ensure a rejected request does not modify the genome.
- Indicate completion and whether the edit succeeded.

Part 4: Module Specification III, Edit Verifier and General Design Requirements

4.1 Module 5: Edit Verifier

Required module name: `<group_id>_edit_verifier`

Objective: Independently verify that the Genome Editor performed the intended substitution without introducing unintended changes.

Signal	Direction	Width	Description
`clk`	Input	1 bit	System clock
`rst`	Input	1 bit	Synchronous active-high reset
`verify_request`	Input	1 bit	Starts verification
`genome_original`	Input	64 bits	Original genome
`genome_edited`	Input	64 bits	Actual edited genome

`edit_position`	Input	5 bits	Starting base index of the intended edit
`replacement`	Input	8 bits	Intended replacement sequence
`verify_pass`	Output	1 bit	Indicates that the actual result matches the expected result
`mismatch_count`	Output	6 bits	Number of differing bases between expected and actual genomes
`done`	Output	1 bit	Indicates verification completion

Functional requirements:

- Independently construct the expected edited genome using the original genome, edit position, and replacement sequence.
- Compare the expected genome against the actual edited genome across all 32 bases.
- Assert `verify_pass` only when the expected and actual genomes match exactly.
- Report the number of differing DNA bases.
- Indicate when verification is complete.
- Reconstruct the expected result independently. Do not rely solely on the Editor's `edit_success` signal.

4.2 General Design Requirements

All five modules must satisfy the following requirements:

- Use **synthesizable Verilog-2001**.
- Use the exact required signal names, widths, and directions.
- Use a rising-edge clock and synchronous active-high reset for sequential modules.
- Avoid unintended latch inference.
- Avoid simulation-only delays in synthesizable logic.
- Handle relevant boundary conditions and invalid requests.
- Remain compatible with the module interfaces specified in this document.

Each team must write its own RTL implementation. No prewritten logic skeletons will be supplied.

Part 5: Testing, Submission and Evaluation

5.1 Testing and Submission

Each team must submit:

1. Verilog source files for all five modules.
2. A testbench for each module.
3. Simulation results demonstrating correct operation.
4. Test cases covering normal operation and relevant edge cases.
5. A brief explanation of each design and its operation.

All files must use the team's assigned group identifier consistently.

5.2 Evaluation

Criterion	Points
Guide Matcher	15
Genome Window Generator	20
Target Locator	20
Genome Editor	20
Edit Verifier	20

Design explanation and test quality	5
Total	**100**

5.3 Final Objective

GENOME-X is a system whose modules must work together reliably. Your task is to build the digital logic that finds an eligible target, performs a controlled edit, and independently verifies the result.

The correctness of your RTL, the quality of your testing, and your ability to explain your design will determine your performance in Round 1.